

WHAT ARE CDES?

Common Data Elements — standardized data fields from the NIH/NLM CDE Repository. Each CDE defines a reusable question, variable, or measurement used across clinical studies.

CDE Record Structure

- `tinyId` — unique short identifier (e.g. "XyZ4k2")
- `designations[]` — alternative names / display labels
- `definitions[]` — explanatory text descriptions
- `valueDomain` — permitted values, data type
- `sources[], properties[]` — provenance metadata

Key Field Paths

<code>designations.*.designation</code>	CDE names
<code>definitions.*.definition</code>	CDE descriptions
<code>valueDomain.permissibleValues.*.valueMeaningName</code>	Value labels
<code>valueDomain.permissibleValues.*.valueMeaningDefinition</code>	Value defs

PYDANTIC DATA MODELS

Defined in `CDE_Schema/`; selected via `-m MODEL`:

Model	Use
CDE	Full CDE records (primary)
Form	CDE forms / questionnaires
EmbedText	Pre-extracted text for embeddings

THREE-LAYER ARCHITECTURE

`cli.py` → `run.py` → `logic/*.py`

CLI	Argument parsing only
Orchestration	File I/O, coordination, formatting
Logic	Core algorithms (stateless, testable)

Entry: `cde_analyzer.py` — lazy-loaded, ~26 actions.

PATTERN TSV FORMAT

Tab-separated files are the interchange format between pipeline steps.

Standard Columns

Column	Description
<code>pattern</code>	Detected text pattern
<code>tinyIds</code>	Pipe-separated CDE ids
<code>type</code>	Detection method
<code>source_pattern</code>	Original form before expansion

Enriched Columns (post field-analysis)

<code>def_count</code>	CDEs matched in definitions
<code>desig_count</code>	CDEs matched in designations
<code>field_profile</code>	<code>def_only</code> , <code>desig_only</code> , or <code>both</code>
<code>example_name</code>	First matching CDE name
<code>example_definition</code>	First matching definition (120 chars)

Hierarchy Columns (coalesced)

<code>group</code>	Parent instrument family
<code>sub_group</code>	Specific sub-domain variant
<code>suffix</code>	Distinguishing suffix text

Strip-Ready Files

- `{base}_full.tsv` — full pattern removal
- `{base}_sub.tsv` — sub-group only (suffix retained)

FILE TYPES REFERENCE

Ext	Content	Used By
<code>.json</code>	CDE records (list of objects)	All actions (<code>-i</code>)
<code>.tsv</code>	Patterns / phrases	Mining, strip, pattern_util
<code>.yaml</code>	Workflows, config files	workflow engine, seeds
<code>.md</code>	Reports (Markdown)	discovery/pipeline_report
<code>.fasta</code>	Lemmatized sequences	lemma_fasta
<code>.csv</code>	Tabular export	extract_embed, count

Workflow state: `.workflow_state.json` — enables `workflow resume`.

CONFIGURATION FILES

File	Purpose
<code>config/temporal_seed_patterns.yaml</code>	25 temporal seed patterns
<code>config/supplementary_patterns.yaml</code>	Global curated instruments
<code>config/anchor_expansions.yaml</code>	Context prefix/suffix expansions
<code>./supplementary_patterns.yaml</code>	Local project override

Config Auto-Discovery

Workflow engine loads

`{output_dir}/{workflow_name}_config.yaml` between YAML defaults and `--set` overrides.

SUPPLEMENTARY PATTERNS

Instrument patterns from external sources or manual curation.

- Global:** `config/supplementary_patterns.yaml`
- Local:** `./supplementary_patterns.yaml` (project override)

```
# Auto-ingest from sanity check
pattern_util --harvest-to-supplementary sanity.tsv
# Promote local → global
pattern_util --promote-supplementary
```

K-MER PHRASE MINING

Iterative descending extraction of shared text phrases from CDE fields.

Algorithm

```

for k = k_max down to k_min:
    phrases = extract_kmers(texts, k)
    phrases = filter(freq ≥ n, tinyids ≥ t)
    emit(phrases)
    texts = mask(texts, phrases) # prevent re-counting

```

Key Parameters

-K	k_max (default 25)	Longest phrase
-k	k_min (default 3)	Shortest phrase
-n	freq_min	Min frequency
-t	min_tinyids	Min distinct CDEs
-D	de Bruijn	Extend via overlap graph
-S	subsumption	Remove subsumed

Masking

After each k, detected phrases replaced with placeholders. Prevents redundant shorter fragments at lower k values.

DEDUP PRE-PASS

Before k-mer mining (Stage 0):

1. **Hash** field texts — group identical strings shared by N+ CDEs
2. **Filter** — only emit phrases with tokens > k_max
3. **No masking** — sub-phrases remain visible to k-mer loop

Output: separate `dedup_phrases.tsv`. Short duplicates found naturally by k-mer mining.

ANCHOR EXPANSION

Context prefix/suffix expansion for instrument discovery:

- Defined in `config/anchor_expansions.yaml`
- Generates “as part of <X>”, “using the <X>”, etc.
- **Instrument-specific only** — phrase/temporal patterns use `\b` word boundaries instead

Phase 1 uses anchor expansion; Phase 2 phrases are self-contained.

SUBSUMPTION

Removing redundant shorter patterns covered by longer ones.

Text-Based Subsumption

Pattern A subsumed by B when:

- A is a **substring** of B, AND
- every tinyld of A is **covered** by union of tinylds of all longer patterns containing A

Empirical Validation

Per-tinyld, per-field verification after coalescing:

- Check if short pattern matches text **not** already covered by any longer group member
- Keep only if it matches **independently**

```

pattern_util -V coalesced.tsv \
-i input.json -o validated.tsv

```

COALESCER PHASES

Three-phase consolidation

(`flexible_pattern_matcher.py`):

Phase 1a — Prefix-Kept

Keep shorter A alongside longer B when

`B.startswith(A)` and A has tinylds not covered by B.

Phase 1b — NP-Continuity

If extension tail is **NP-like** (80%+ Title Case words), keep B intact — preserves instrument sub-domain names.

```

# Kept: "Neuro-QOL Sleep Disturbance"
# Rolled down: "PHQ-9, which is a"

```

Phase 2 — Prefix Trie

Group patterns sharing common prefix. Merge groups where shorter prefix covers same tinylds.

TEMPORAL SEED PATTERNS

25 seeds in `config/temporal_seed_patterns.yaml`:

Category	Examples
Recency	In the past {N} {days weeks months}
Duration	For {N} {years months weeks}
Frequency	{N} times per {week month year}
Specific	In the past 7 days, 48 hours

TEMPORAL VARIANT EXPANSION

Each seed expands via 4 generators:

- **Temporal prepositions:** In the past / Over the last / During the previous
- **Case:** lowercase, Title Case
- **Number:** singular ↔ plural units
- **Article:** bare, with “the”, with “a”

25 seeds → ~2,100 expanded variants

```

pattern_util -T -o temporal.tsv

```

SPLIT TEMPORAL / CURATED

Phase 3 branching pipeline uses **two separate passes**:

Temporal → **Curated**

Pass	Case	Patterns
1. Temporal	Case-insensitive (<code>-I</code>)	~2,100 auto-generated
2. Curated	Case-sensitive	Human-curated phrases

Temporal uses `re.IGNORECASE`; curated uses verbatim expansion for controlled matching.